

STAT 427: Illinois Campus Cluster Usage Analysis

Rushil Bhupta Jiachen Cong Vladislav Fedorov

Rishab Sakalkale Yanlin Wang

Outline

- 1 Project Overview
- 2 Job submission behavior & execution schedules - Jiachen, Yanlin
- 3 Resource imbalance & GPU extension analysis - Rushil
- 4 Types of workloads conducted on the cluster - Rishab
- 5 Cluster Reservation/Usage - Vlad

Outline

- 1 Project Overview
- 2 Job submission behavior & execution schedules - Jiachen, Yanlin
- 3 Resource imbalance & GPU extension analysis - Rushil
- 4 Types of workloads conducted on the cluster - Rishab
- 5 Cluster Reservation/Usage - Vlad

Problem Statement and Project Goals

Problem:

- The Illinois Campus Cluster is a shared high-performance computing system used by researchers across campus.
- Despite its widespread use, cluster usage patterns and system efficiency are not well understood.

Project Goals:

- Identify when and how jobs are submitted and executed
- Evaluate whether resources are used efficiently across users and departments
- Characterize dominant workload types on the cluster
- Provide insights to support scheduling and resource allocation decisions

Breakdown of Analyses

We have been working on independent analyses of the data, whose high-level summaries are listed below:

- ① **Job submission behavior and execution schedules** $\Rightarrow$ Jiachen, Yanlin
- ② **User demographics and departmental usage patterns** $\Rightarrow$ Rushil
- ③ **Types of workloads conducted on the cluster** $\Rightarrow$ Rishab
- ④ **Timing patterns of interactive versus batch jobs** $\Rightarrow$ Vlad

Outline

- 1 Project Overview
- 2 Job submission behavior & execution schedules - Jiachen, Yanlin
- 3 Resource imbalance & GPU extension analysis - Rushil
- 4 Types of workloads conducted on the cluster - Rishab
- 5 Cluster Reservation/Usage - Vlad

Data Processing Pipeline (whole-cluster-usage/)

Input Data ($\approx 5\text{GB}$)

- 12 monthly accounting logs
- 120 columns, 6,059,140 rows (after merging)

Preprocessing and Feature Construction

- Restrict jobs with Submit year = 2025
- Convert "", "unknown", "none", and "n/a" to NA
- Keep analysis-relevant columns (e.g., ReqMem, ReqCPUS)
- Construct 3 variables: ReqMem_MB, Waittime, Runtime

Output whole_cluster_usage_merged_raw.txt.gz ($\approx 250\text{MB}$)



Job-Level Dataset Construction

Problem

- A single job may appear in multiple rows
- Direct analysis would lead to **double-counting**

Solution

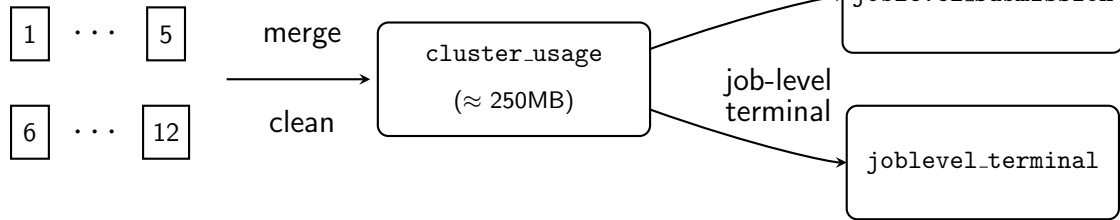
- Construct job-level datasets with **one row per job**
- De-duplicate by JobIDRaw (hash-based bucketing)
- Select representative records for each job

Two analytical views

- **Submission view**: earliest snapshot → 6,025,326 rows × 16 columns
- **Terminal view**: final snapshot → 6,025,326 rows × 24 columns

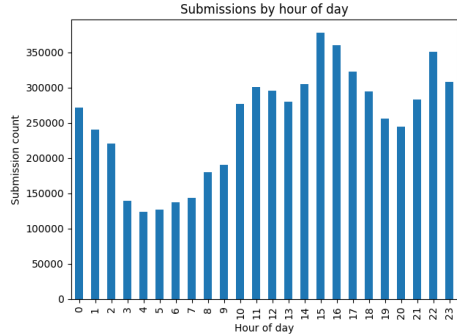
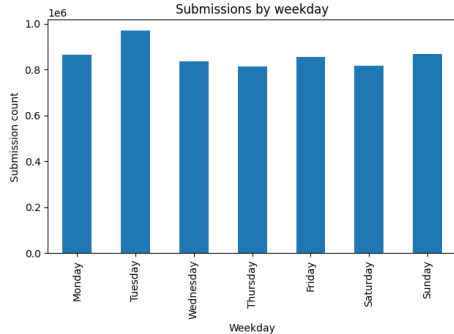
Data Construction Pipeline and Final Datasets

12 monthly logs ($\approx 5\text{GB}$)



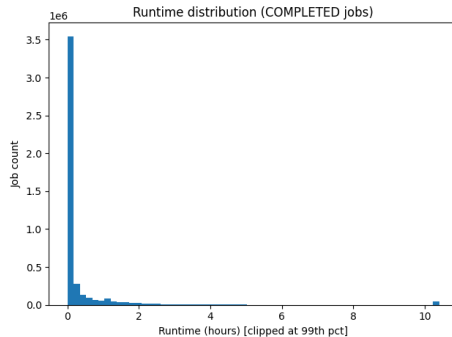
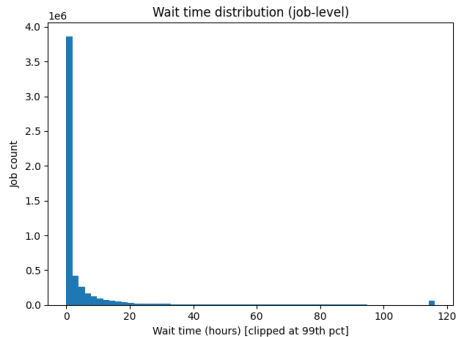
Our downstream analysis mainly focuses on the **submission** and **terminal** job-level datasets.

When Are Jobs Submitted?



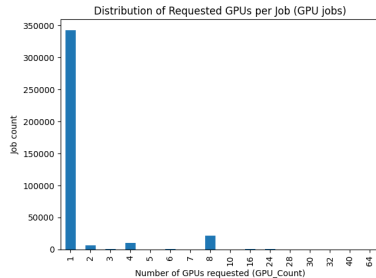
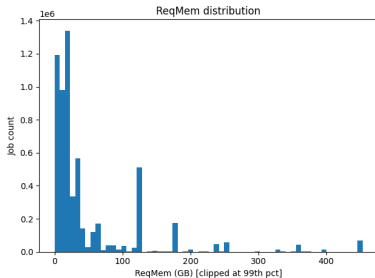
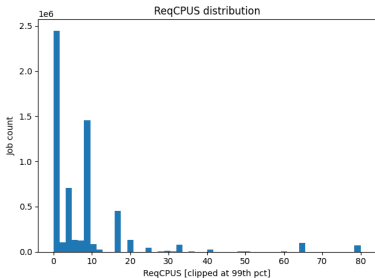
- Job submissions are relatively consistent across weekdays (however, more submissions at the beginning of a week)
- Submissions peak in the afternoon and late evening hours; lowest in early morning; a slight drop in evening (7-8 P.M.)

WaitingTime & Runtime



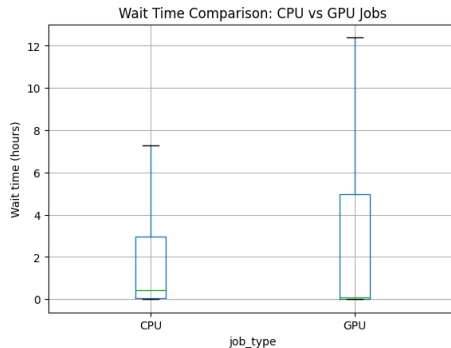
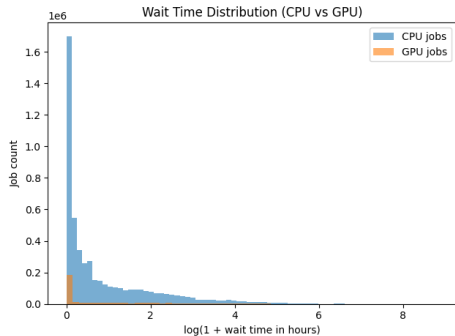
- Waiting time is highly skewed with a long tail; most jobs don't wait long; some statistics (in hour): Median: 0.42; 90th Percentile: 13.01.
- Most jobs run very quickly (median runtime ≈ 0.02 hours). However, the runtime distribution is highly right-skewed. A small number of jobs run for a long time (99th percentile ≈ 10 hours).

Resource Demand Across Jobs: CPU, Mem & GPU



- Most jobs request small CPU (median: 4 CPUs; 90th percentile: 16 CPUs) and memory (median: 16 GB; 90th percentile: 125GB) resources
- About 89% of GPU jobs request only one GPU, while only about 6% of jobs request five or more GPUs. However, there are some large GPU requests (e.g. 40 GPUs, 64 GPUs).

GPU Jobs Experience Longer Waiting Times



- **Method:** Mann–Whitney U test
- **Result:** The p-value is **approximately 0** (< 0.05). Reject the null hypothesis.
- **Conclusion:** CPU and GPU waiting times are **significantly different**.
- GPU jobs have **longer and more variable waiting times**
- Likely due to **limited GPU resources and high demand**

Predicting GPU Job Waiting Time - Stage 1: Classification

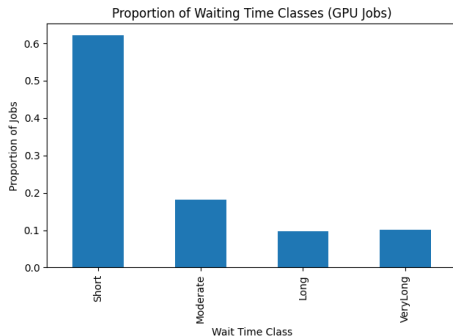
Two-stage modeling idea: classification first, then more precise prediction within each class.

Define 4 waiting-time categories

- **Short:** < 1 hour
- **Moderate:** 1 – 8 hours
- **Long:** 8 – 24 hours
- **VeryLong:** > 24 hours

Modeling procedure

- **multinomial logistic regression** model
- **Time features:** submission weekday, submission hour
- **Scheduling features:** QOS, priority
- **Resource features:** GPU count, requested CPUs, nodes, memory, and time limit



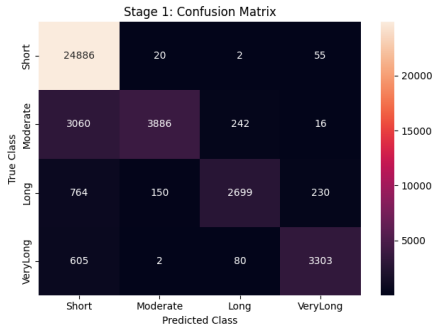
Predicting GPU Job Waiting Time - Stage 2: Precise Prediction

Modeling procedure

- Use **Linear Regression** model
- We use the same set of features as in Stage 1.
- Separate regression model for each waiting-time category
- Predict exact waiting time within each category

Prediction Result - Stage 1

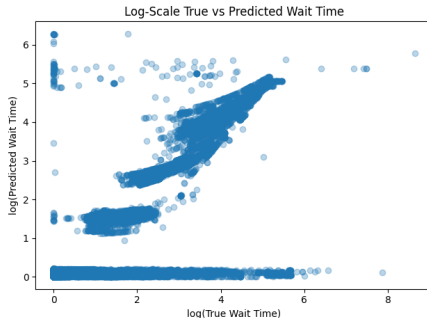
- Train/test split: 80% / 20%
- The overall accuracy of the classification model is about 87%.



- The model performs very well for Short and Very Long jobs, but Moderate jobs are often misclassified as Short. $\Rightarrow$ Consider redefining the classification thresholds.

Prediction Result - Stage 2

- Metric: RMSLE.
- $\text{RMSLE}=0.77$; which suggests that the predicted waiting time is typically within a factor of about 2 of the true waiting time.
- The model captures the overall waiting-time trend well on the log scale, but tends to underestimate very long waits and some moderate waits.



Future Directions

- Refining the definitions of some important thresholds for classification models in Stage 1.
- Using different prediction models in Stage 2.
- Building a waiting-time prediction model for CPU-only jobs.
- Exploring alignment between submission patterns and external deadlines (e.g., conference or journal deadlines)

Outline

- 1 Project Overview
- 2 Job submission behavior & execution schedules - Jiachen, Yanlin
- 3 Resource imbalance & GPU extension analysis - Rushil**
- 4 Types of workloads conducted on the cluster - Rishab
- 5 Cluster Reservation/Usage - Vlad

This analysis focuses on understanding **resource utilization patterns** in the campus cluster

Key areas explored:

- **User-level workload distribution**
- **Department-wise resource consumption**
- **CPU usage patterns and imbalance**

item Extended the analysis to include **GPU usage**, which is not directly available in the dataset set item GPU usage was extracted from system-level fields to analyze:

- **GPU vs CPU workload distribution**
- **GPU access inequality across users**

Heavy User Analysis

- Significant **imbalance in job submissions**
- Top user alone contributes **~20% of total workload**
- Small group of users dominate cluster usage
- Indicates **heavy-user dependency on system**

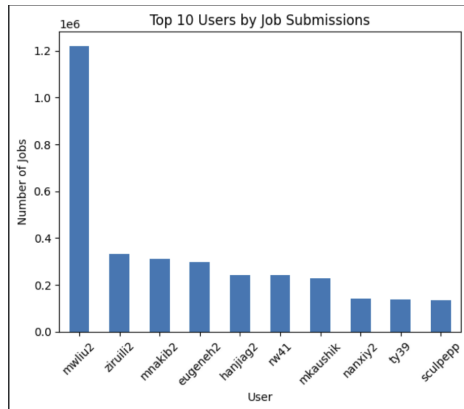


Figure 1: Top 10 Users by Job Submission

Top Department Analysis

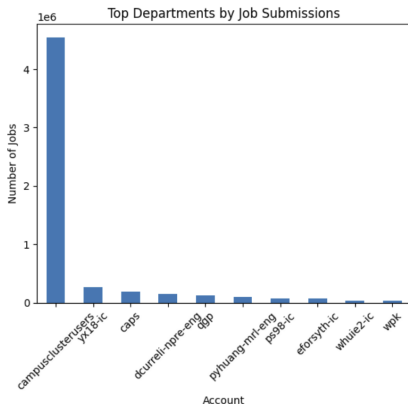


Figure 2: Top Departments By Job Submission

- Job submissions are highly **concentrated in a few departments**
- campusclusterusers dominates overall workload
- These same departments also **dominate CPU usage**
- Indicates strong **centralization of resource consumption**

CPU Workload Distribution

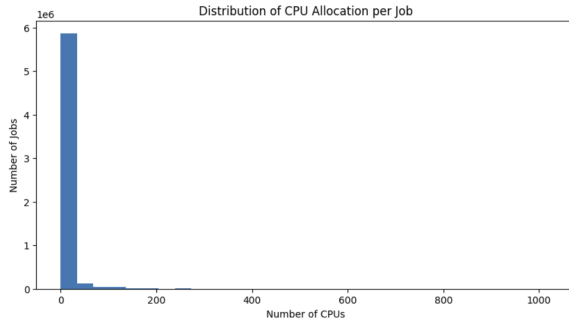


Figure 3: CPU Allocation per Job

- Majority of jobs use **low CPU resources**
- Small number of jobs consume **very high CPUs**
- Indicates **long-tail workload distribution**
- High CPU jobs may create **scheduling bottlenecks**

CPU vs GPU Usage Gap

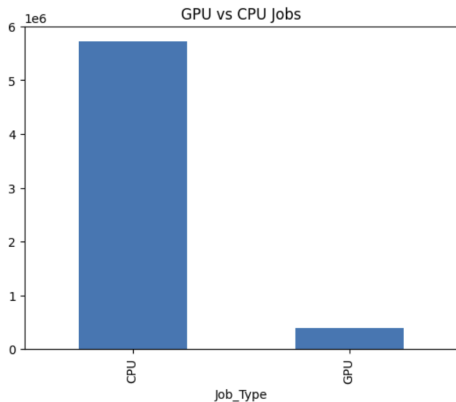


Figure 4: CPU vs GPU Job Distribution

GPU Jobs: 389133

CPU Jobs: 5716185

- GPU jobs form only **~6% of total workload**
- CPU dominates cluster usage
- Indicates **underutilization of GPU resources**
- Suggests potential for **better GPU adoption**

GPU Usage Concentration Across Users

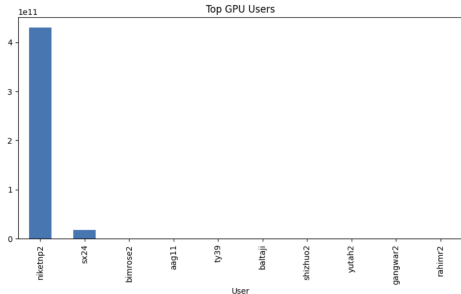


Figure 5: Top GPU users

- GPU usage is **extremely concentrated among a small group of users**
- Majority of users have **little to no access to GPU resources**
- Indicates **significant imbalance in resource allocation**
- Suggests GPUs are primarily used by **specialized workloads or users**

Top 10% of users account for **nearly all GPU usage (~100%)**

Key Findings & Future Directions

Key Findings:

- User workload is **highly skewed**
- Department usage is **strongly centralized**
- CPU resources are **widely utilized across jobs**
- GPU resources:
 - **Underutilized overall**
 - **Highly concentrated among few users**

Key Questions:

- Why is GPU usage limited to a small group of users?
- Are GPU resources **accessible to all users fairly?**
- How can GPU utilization be **improved without affecting fairness?**

Future Directions:

- Improve **GPU accessibility**
- Monitor **heavy users and resource consumption**

Outline

- 1 Project Overview
- 2 Job submission behavior & execution schedules - Jiachen, Yanlin
- 3 Resource imbalance & GPU extension analysis - Rushil
- 4 Types of workloads conducted on the cluster - Rishab
- 5 Cluster Reservation/Usage - Vlad

Job States - Rishab

outcome	job_count	job_pct	row_count	row_pct
Cancelled	639041	10.59	641112	10.50
Completed	4644274	76.98	4662342	76.36
Failed	585157	9.70	586554	9.61
Node Failed	218	0.00	220	0.00
Other	845	0.01	850	0.01
Out of Memory	43313	0.72	43546	0.71
Requeued	9	0.00	9	0.00
Running	301	0.00	3600	0.06
Stuck PENDING	3791	0.06	45464	0.74
Timeout	116144	1.93	122413	2.00
Total unique jobs: 6033093				
Total rows: 6106110				

Figure 6: Job State Counts

- 76% of jobs are completed
- 43k jobs are out-of-memory
- 20% of jobs are either cancelled or failed

Takeaway: System failures account for a fraction of incomplete jobs, while job errors (cancellations, failed jobs) are significantly higher. How can we minimize this?

Final State Volume Over Time

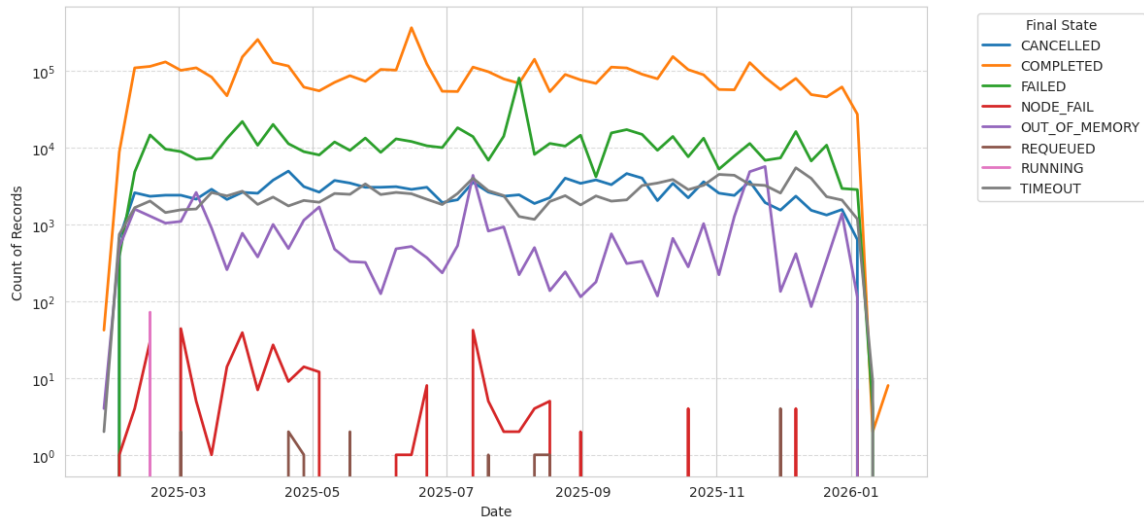


Figure 7: Job State Counts Over Time

Job Records Contain Terminal States - Rishab

	count	mean	std	min	25%	50%	75%	max
outcome								
Cancelled	639041.0	1.003241	0.065675	1.0	1.0	1.0	1.0	10.0
Completed	4644274.0	1.003890	0.062251	1.0	1.0	1.0	1.0	2.0
Failed	585157.0	1.002387	0.048943	1.0	1.0	1.0	1.0	3.0
Node Failed	218.0	1.009174	0.095562	1.0	1.0	1.0	1.0	2.0
Other	845.0	1.005917	0.076741	1.0	1.0	1.0	1.0	2.0
Out of Memory	43313.0	1.005379	0.073148	1.0	1.0	1.0	1.0	2.0
Requeued	9.0	1.000000	0.000000	1.0	1.0	1.0	1.0	1.0
Running	301.0	11.960133	0.267841	10.0	12.0	12.0	12.0	12.0
Stuck PENDING	3791.0	11.992614	0.112296	7.0	12.0	12.0	12.0	12.0
Timeout	116144.0	1.053976	0.237671	1.0	1.0	1.0	1.0	3.0

Figure 8: Job State Counts

- 1 unique job = 1 row in the SLURM dataset
- Only pending and running states have multiple rows per job

Pending Jobs - Rishab

Why are 3791 jobs stuck pending?

- 88.7% only requests 1 CPU
- 70% of jobs requested by 1 user in the “astro-physics” group

Are there factors that leave a job stuck on the queue?

Jobs, Allocated CPUs, Allocated Memory over time - Rishab

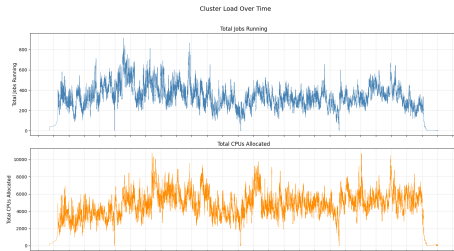


Figure 9: Job State Counts

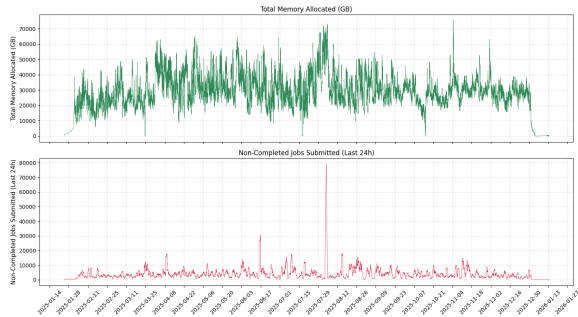


Figure 10: Job State Counts

Next Steps - Rishab

- Can we predict, based on cluster-wide statistics and job requests, whether a job will not complete?
 - I.e. is there a correlation between ReqMem and total memory used across the cluster and node failure / cancelled jobs?
- Do we need to explore queue rules that leave jobs pending?
- Do certain users/accounts/partitions have high failure rates?
 - Could indicate users who need guidance on job configuration

Outline

- 1 Project Overview
- 2 Job submission behavior & execution schedules - Jiachen, Yanlin
- 3 Resource imbalance & GPU extension analysis - Rushil
- 4 Types of workloads conducted on the cluster - Rishab
- 5 Cluster Reservation/Usage - Vlad

What are we trying to understand?

- How do requested runtimes compare to actual runtimes?
- Does overestimation matter operationally?

Why we find this important

- Cluster efficiency
- Job throughput
- Effect on user wait times

Variables of Interest:

- ElapsedRaw (originally in seconds)
- TimeLimitRaw (originally in minutes)
- Partition

Relevant EDA:

- Standardization (minutes to seconds)
- Only jobs with nonzero ElapsedRaw

Key Findings

Massive runtime overestimation across the cluster

- Median runtime: 1.2 minutes
- Median requested: 240 minutes (4 hours)
- Median utilization: 0.57%
- 95.5% of jobs use less than 50% of requested time

Scale of Inefficiency

- Roughly 800 million CPU-hours reserved but unused

Overestimation persists even for longer workloads After filtering out short jobs:

- 5 min jobs - 11.1x overestimation
- 10 min jobs - 7.5x
- 30 min jobs - 3.8x

Short jobs inflate extremes, but behavior is systematic across all job sizes

User behavior and system design

- Poor user estimation of true runtime
- No penalty for overestimation
- Incentive to avoid job failure

System constraints:

- CPUs are hard-reserved
- No reallocation of unused CPUs mid-job
- Overestimation leads to fewer jobs being able to run simultaneously

Potential Ways:

- Improve runtime guidance
 - Historical data,
 - Educational resources, or
 - Lightweight script-based estimation
- Encourage smaller and more accurate time requests
- Introduce automatic runtime extensions
 - Removes the need to contact admins
 - Should lead to a generally lower initial requested time

Thank you for listening!

Questions?